

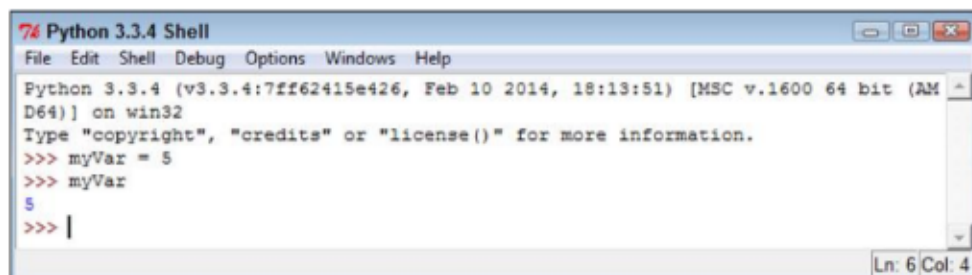
Defining the Essential Python Data Types

Every programming language defines variables that hold specific kinds of information, and Python is no exception. The specific kind of variable is called a *data type*. Knowing the data type of a variable is important because it tells you what kind of information you find inside. In addition, when you want to store information in a variable, you need a variable of the correct data type to do it. Python doesn't allow you to store text in a variable designed to hold numeric information. Doing so would damage the text and cause problems with the application. You can generally classify Python data types as numeric, string, and Boolean, although there really isn't any limit on just how you can view them. The following sections describe each of the standard Python data types within these classifications.

Putting information into variables

To place a value into any variable, you make an assignment using the assignment operator (=). Chapter 6 discusses the whole range of basic Python operators in more detail, but you need to know how to use this particular operator to some extent now. For example, to place the number 5 into a variable named `myVar`, you type **`myVar = 5`** and press Enter at the Python prompt. Even though Python doesn't provide any additional information to you, you can always type the variable name and press Enter to see the value it contains, as shown in Figure 5-1.

Figure 5-1:
Use the assignment operator to place information into a variable.



```
Python 3.3.4 Shell
File Edit Shell Debug Options Windows Help
Python 3.3.4 (v3.3.4:7ff62415e426, Feb 10 2014, 18:13:51) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> myVar = 5
>>> myVar
5
>>> |
```

Understanding the numeric types

Humans tend to think about numbers in general terms. We view 1 and 1.0 as being the same number — one of them simply has a decimal point. However, as far as we're concerned, the two numbers are equal and we could easily use them interchangeably. Python views them as being different kinds of numbers

Understanding Python data types and operator precedence

because each form requires a different kind of processing. The following sections describe the integer, floating-point, and complex number classes of data types that Python supports.

Integers

Any whole number is an *integer*. For example, the value 1 is a whole number, so it's an integer. On the other hand, 1.0 isn't a whole number; it has a decimal part to it, so it's not an integer. Integers are represented by the `int` data type.

As with storage boxes, variables have capacity limits. Trying to stuff a value that's too large into a storage box results in an error. On most platforms, you can store numbers between $-9,223,372,036,854,775,808$ and $9,223,372,036,854,775,807$ within an `int` (which is the maximum value that fits in a 64-bit variable). Even though that's a really large number, it isn't infinite.

When working with the `int` type, you have access to a number of interesting features. Many of them appear later in the book, but one feature is the ability to use different numeric bases:

- ✓ **Base 2:** Uses only 0 and 1 as numbers.
- ✓ **Base 8:** Uses the numbers 0 through 7.
- ✓ **Base 10:** Uses the usual numeric system.
- ✓ **Base 16:** Is also called *hex* and uses the numbers 0 through 9 and the letters A through F to create 16 different possible values.

To tell Python when to use bases other than base 10, you add a 0 and a special letter to the number. For example, `0b100` is the value one-zero-zero in base 2. Here are the letters you normally use:

- ✓ **b:** Base 2
- ✓ **o:** Base 8
- ✓ **x:** Base 16

It's also possible to convert numeric values to other bases using the `bin()`, `oct()`, and `hex()` commands. So, putting everything together, you can see how to convert between bases using the commands shown in Figure 5-2. Try the command shown in the figure yourself so that you can see how the various bases work. Using a different base actually makes things easier in many situations, and you'll encounter some of those situations later in the book. For now, all you really need to know is that integers support different numeric bases.

Understanding Python data types and operator precedence

Floating-point values

Any number that includes a decimal portion is a floating-point value. For example, 1.0 has a decimal part, so it's a floating-point value. Many people get confused about whole numbers and floating-point numbers, but the difference is easy to remember. If you see a decimal in the number, then it's a floating-point value. Python stores floating-point values in the `float` data type.

Floating-point values have an advantage over integer values in that you can store immensely large or incredibly small values in them. As with integer variables, floating-point variables have a storage capacity. In their case, the maximum value that a variable can contain is $\pm 1.7976931348623157 \times 10^{308}$ and the minimum value that a variable can contain is $\pm 2.2250738585072014 \times 10^{-308}$ on most platforms.

When working with floating-point values, you can assign the information to the variable in a number of ways. The two most common methods are to provide the number directly and to use scientific notation. When using scientific notation, an *e* separates the number from its exponent. Figure 5-3 shows both methods of making an assignment. Notice that using a negative exponent results in a fractional value.

Complex numbers

You may or may not remember complex numbers from school. A *complex number* consists of a real number and an imaginary number that are paired together. Just in case you've completely forgotten about complex numbers, you can read about them at <http://www.mathsisfun.com/numbers/complex-numbers.html>. Real-world uses for complex numbers include:

- ✓ Electrical engineering
- ✓ Fluid dynamics
- ✓ Quantum mechanics
- ✓ Computer graphics
- ✓ Dynamic systems

Complex numbers have other uses, too, but this list should give you some ideas. In general, if you aren't involved in any of these disciplines, you probably won't ever encounter complex numbers. However, Python is one of the few languages that provides a built-in data type to support them. As you progress through the book, you find other ways in which Python lends itself especially well to science and engineering.

The imaginary part of a complex number always appears with a *j* after it. So, if you want to create a complex number with 3 as the real part and 4 as the imaginary part, you make an assignment like this:

```
myComplex = 3 + 4j
```

If you want to see the real part of the variable, you simply type **myComplex.real** at the Python prompt and press Enter. Likewise, if you want to see the imaginary part of the variable, you type **myComplex.imag** at the Python prompt and press Enter.

Understanding Boolean values

It may seem amazing, but computers always give you a straight answer! A computer will never provide “maybe” as output. Every answer you get is either `True` or `False`. In fact, there is an entire branch of mathematics called Boolean algebra that was originally defined by George Boole (a super-geek of his time) that computers rely upon to make decisions. Contrary to common belief, Boolean algebra has existed since 1854 — long before the time of computers.

Understanding Python data types and operator precedence

When using Boolean value in Python, you rely on the `bool` type. A variable of this type can contain only two values: `True` or `False`. You can assign a value by using the `True` or `False` keywords, or you can create an expression that defines a logical idea that equates to true or false. For example, you could say, `myBool = 1 > 2`, which would equate to `False` because 1 is most definitely not greater than 2. You see the `bool` type used extensively in the book, so don't worry about understanding this concept right now.

Understanding strings

Of all the data types, strings are the most easily understood by humans and not understood at all by computers. If you have read the previous chapters in this book, you have already seen strings used quite. For example, all the example code in Chapter 4 relies on strings. A *string* is simply any grouping of characters you place within double quotes. For example, `myString = "Python is a great language."` assigns a string of characters to `myString`.

The computer doesn't see letters at all. Every letter you use is represented by a number in memory. For example, the letter *A* is actually the number 65. To see this for yourself, type `ord("A")` at the Python prompt and press Enter. You see 65 as output. It's possible to convert any single letter to its numeric equivalent using the `ord()` command.

Because the computer doesn't really understand strings, but strings are so useful in writing applications, you sometimes need to convert a string to a number. You can use the `int()` and `float()` commands to perform this conversion. For example, if you type `myInt = int("123")` and press Enter at the Python prompt, you create an `int` named `myInt` that contains the value 123. Figure 5-4 shows how you can perform this task and validate the content and type of `myInt`.

You can convert numbers to a string as well by using the `str()` command. For example, if you type `myStr = str(1234.56)` and press Enter, you create a string containing the value `"1234.56"` and assign it to `myStr`. Figure 5-5 shows this type of conversion and the test you can perform on it. The point is that you can go back and forth between strings and numbers with great ease. Later chapters demonstrate how these conversions make a lot of seemingly impossible tasks quite doable.

Understanding Python data types and operator precedence

Working with Dates and Times

Dates and times are items that most people work with quite a bit. Society bases almost everything on the date and time that a task needs to be or was completed. We make appointments and plan events for specific dates and times. Most of our day revolves around the clock. Because of the time-oriented nature of humans, it's a good idea to look at how Python deals with interacting with dates and time (especially storing these values for later use). As with everything else, computers understand only numbers — the date and time don't really exist.



To work with dates and times, you need to perform a special task in Python. When writing computer books, chicken-and-egg scenarios always arise, and this is one of them. To use dates and times, you must issue a special import

`datetime` command. Technically, this act is called *importing a module*, and you learn more about it in Chapter 10. Don't worry how the command works right now — just use it whenever you want to do something with date and time.

Computers do have clocks inside them, but the clocks are for the humans using the computer. Yes, some software also depends on the clock, but again, the emphasis is on human needs rather than anything the computer might require. To get the current time, you can simply type **`datetime.datetime.now()`** and press Enter. You see the full date and time information as found on your computer's clock (see Figure 5-6).

```
Python 3.3.4 Shell
File Edit Shell Debug Options Windows Help
Python 3.3.4 (v3.3.4:7ff62415e426, Feb 10 2014, 18:13:51) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> import datetime
>>> datetime.datetime.now()
datetime.datetime(2014, 3, 8, 19, 28, 12, 988250)
>>>
```

You may have noticed that the date and time are a little hard to read in the existing format. Say that you want to get just the current date, in a readable format. It's time to combine a few things you discovered in previous sections to accomplish that task. Type **`str(datetime.datetime.now().date())`** and press Enter. Figure 5-7 shows that you now have something a little more usable.

```
Python 3.3.4 Shell
File Edit Shell Debug Options Windows Help
Python 3.3.4 (v3.3.4:7ff62415e426, Feb 10 2014, 18:13:51) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> import datetime
>>> datetime.datetime.now()
datetime.datetime(2014, 3, 8, 19, 28, 12, 988250)
>>> str(datetime.datetime.now().date())
'2014-03-08'
>>>
```

Interestingly enough, Python also has a `time()` command, which you can use to obtain the current time. You can obtain separate values for each of the components that make up date and time using the day, month, year, hour, minute, second, and microsecond values. Later chapters help you understand how to use these various date and time features to keep application users informed about the current date and time on their system.

Understanding Python data types and operator precedence

Understanding operator precedence

When you create simple statements that contain just one operator, the order of determining the output of that operator is also simple. However, when you start working with multiple operators, it becomes necessary to determine which operator to evaluate first. For example, it's important to know whether $1 + 2 * 3$ evaluates to 7 (where the multiplication is done first) or 9 (where the addition is done first). An order of operator precedence tells you that the answer is 7 unless you use parentheses to override the default order. In this case, $(1 + 2) * 3$ would evaluate to 9 because the parentheses have a higher order of precedence than multiplication does. Table 6-9 defines the order of operator precedence for Python.

Table 6-9 Python Operator Precedence	
Operator	Description
<code>()</code>	You use parentheses to group expressions and to override the default precedence so that you can force an operation of lower precedence (such as addition) to take precedence over an operation of higher precedence (such as multiplication).
<code>**</code>	Exponentiation raises the value of the left operand to the power of the right operand.
<code>~ + -</code>	Unary operators interact with a single variable or expression.
<code>* / % //</code>	Multiply, divide, modulo, and floor division.
<code>+ -</code>	Addition and subtraction.
<code>>> <<</code>	Right and left bitwise shift.
<code>&</code>	Bitwise AND.
<code>^ </code>	Bitwise exclusive OR and standard OR.
<code><= < > >=</code>	Comparison operators.
<code>== !=</code>	Equality operators.
<code>= %= /= //= -= += *= **=</code>	Assignment operators.
<code>is</code>	Identity operators.
<code>is not</code>	
<code>in</code>	Membership operators.
<code>not in</code>	
<code>not or and</code>	Logical operators.

Understanding Python data types and operator precedence

Understanding operator precedence

When you create simple statements that contain just one operator, the order of determining the output of that operator is also simple. However, when you start working with multiple operators, it becomes necessary to determine which operator to evaluate first. For example, it's important to know whether $1 + 2 * 3$ evaluates to 7 (where the multiplication is done first) or 9 (where the addition is done first). An order of operator precedence tells you that the answer is 7 unless you use parentheses to override the default order. In this case, $(1 + 2) * 3$ would evaluate to 9 because the parentheses have a higher order of precedence than multiplication does. Table 6-9 defines the order of operator precedence for Python.

Table 6-9 Python Operator Precedence	
Operator	Description
<code>()</code>	You use parentheses to group expressions and to override the default precedence so that you can force an operation of lower precedence (such as addition) to take precedence over an operation of higher precedence (such as multiplication).
<code>**</code>	Exponentiation raises the value of the left operand to the power of the right operand.
<code>~ + -</code>	Unary operators interact with a single variable or expression.
<code>* / % //</code>	Multiply, divide, modulo, and floor division.
<code>+ -</code>	Addition and subtraction.
<code>>> <<</code>	Right and left bitwise shift.
<code>&</code>	Bitwise AND.
<code>^ </code>	Bitwise exclusive OR and standard OR.
<code><= < > >=</code>	Comparison operators.
<code>== !=</code>	Equality operators.
<code>= %= /= //= -= += *= **=</code>	Assignment operators.
<code>is</code>	Identity operators.
<code>is not</code>	
<code>in</code>	Membership operators.
<code>not in</code>	
<code>not or and</code>	Logical operators.

Understanding Python data types and operator precedence

Table 6-2 Python Arithmetic Operators		
<i>Operator</i>	<i>Description</i>	<i>Example</i>
+	Adds two values together	5 + 2 = 7
-	Subtracts the right operand from the left operand	5 - 2 = 3
*	Multiplies the right operand by the left operand	5 * 2 = 10
/	Divides the left operand by the right operand	5 / 2 = 2.5
%	Divides the left operand by the right operand and returns the remainder	5 % 2 = 1
**	Calculates the exponential value of the right operand by the left operand	5 ** 2 = 25
//	Performs integer division, in which the left operand is divided by the right operand and only the whole number is returned (also called floor division)	5 // 2 = 2

Relational

The relational operators compare one value to another and tell you when the relationship you've provided is true. For example, 1 is less than 2, but 1 is never greater than 2. The truth value of relations is often used to make decisions in your applications to ensure that the condition for performing a specific task is met. Table 6-3 describes the relational operators.

Table 6-3 Python Relational Operators		
<i>Operator</i>	<i>Description</i>	<i>Example</i>
==	Determines whether two values are equal. Notice that the relational operator uses two equals signs. A mistake many developers make is using just one equals sign, which results in one value being assigned to another.	1 == 2 is False
!=	Determines whether two values are not equal. Some older versions of Python allowed you to use the <> operator in place of the != operator. Using the <> operator results in an error in current versions of Python.	1 != 2 is True
>	Verifies that the left operand value is greater than the right operand value.	1 > 2 is False